Guided Flow Matching for Extreme Weather Storylines

Master's Thesis

Author

Alain Joss

alain.joss@uzh.ch

Supervisors

Maxim Samarin

Shirin Goshtasbpour

Prof. Dr. Guillaume Obozinski

Swiss Data Science Center

Date of Submission: August 12, 2026

Abstract

Contents

List of Figures

List of Tables

Chapter 1

Introduction

Machine-learning models have reached state-of-the-art skill in weather forecasting, rivaling traditional physics-based numerical models at a fraction of their computational cost [14, 3]. More recently, generative models have extended these advances to probabilistic forecasting [20, 6]. Instead of producing a single deterministic forecast, they are trained to approximate the conditional distribution of future atmospheric states given the available atmospheric state and can therefore generate multiple plausible future trajectories [20]. This allows them to represent forecast uncertainty explicitly, alleviating the smoothing effects often observed in deterministic neural forecasts, enabling direct ensemble generation, and achieving strong probabilistic skill as measured, for instance, by the Continuous Ranked Probability Score (CRPS) [10, 20].

Beyond their use as probabilistic samplers, generative models can also be actively steered during generation. In image generation, applications such as text-to-image generation, editing, and inpainting routinely rely on *guidance* to steer a pretrained generative model toward a desired outcome [7, 12, 19]. Generative weather models, by contrast, are still predominantly used in their unguided form, while their potential for controlled generation remains comparatively unexplored. Only a small number of recent studies have investigated related forms of weather control, including classifier-guided generation of tropical cyclones with Climate in a Bottle [4], precipitation-reduction interventions through guided diffusion [26], and optimized heatwave storylines using differentiable weather models [28].

This opens up a complementary use of generative weather models beyond probabilistic forecasting. While forecasting asks which future weather trajectories are likely given the current state, *scenario generation* asks what a prescribed alternative evolution could look like while remaining meteorologically plausible [23]. This distinction is particularly relevant for extreme weather. Extreme events account for a disproportionate share of weather-related impacts [29],

yet their rarity means that both historical observations and finite forecast ensembles provide only limited examples from which to study them. The historical record necessarily contains only a finite sample of possible extremes, while risk analysis may also require considering plausible alternatives, including scenarios more severe than those previously observed [25, 23].

We are therefore interested in exploiting the ability of generative weather models to produce coherent weather trajectories while deliberately steering them toward prescribed extremes. Rather than generating an isolated extreme state, the objective is to obtain an event that remains structured as weather throughout its evolution: coherent over time and space, across atmospheric pressure levels, and between the different meteorological variables. In other words, the imposed extreme should remain embedded within a multivariate atmospheric evolution that is consistent with the structure of realistic weather.

Transferring guidance methods from image generation to this setting is not immediate. First, the conditioning objective is different. Image-generation guidance commonly targets semantic properties, whereas a weather scenario can be specified quantitatively by prescribing the evolution of selected atmospheric quantities over a spatial domain and forecast horizon [7, 12, 19]. Second, plausibility is considerably more structured. Satisfying such a target is not sufficient on its own: the generated trajectory must remain coherent beyond the prescribed quantities and spatial domain, across neighboring locations, atmospheric variables, pressure levels, and subsequent forecast steps. Third, the role of guidance strength changes. In image generation, guidance strength is commonly used to control the trade-off between stronger conditioning and sample fidelity or diversity [7, 12]. For weather scenario generation, an explicit quantitative target instead gives us a measurable objective: the model should be guided sufficiently to reach the prescribed trajectory, while the consequences of that guidance on the remaining forecast must be evaluated explicitly.

These considerations lead to the central research question of this thesis:

> *Can we steer generative probabilistic weather models to produce extreme weather trajectories that satisfy a prescribed target while remaining meteorologically plausible?*

We address this question through a general framework composed of three stages: *Specify*, *Guide*, and *Evaluate*. The first determines the target trajectory toward which the model should be steered. The second modifies the generative process to reach that target. The third assesses both whether the prescribed event has been realized and whether the resulting trajectory remains consistent with the structure of realistic weather.

We investigate this framework using ArchesWeather [6], a probabilistic weather model based on flow matching. As an experimental case study, we focus on regional surface-temperature heatwaves and formulate the guidance objective in terms of a prescribed temperature trajectory over a chosen region. These choices define the particular scenario studied in this thesis rather than the methodology itself: the guidance framework operates on differentiable objectives defined on the forecast state and can therefore, in principle, be used to prescribe different meteorological quantities and forms of weather evolution.

The three stages of the framework correspond to the main contributions of this thesis:

1. **Specify.** We formalize the desired scenario as a quantitative target trajectory of a selected atmospheric quantity over a prescribed spatial region. We consider two complementary constructions. For controlled evaluation of the guidance method, targets are defined as prescribed relative deviations from a matched unguided forecast. For storyline generation, targets are constructed from data-driven historical weather trajectories, providing complete temporal evolutions toward which the model can be steered.
2. **Guide.** We develop a target-driven, gradient-based guidance method for flow-matching weather models. Rather than selecting a fixed guidance strength and accepting the resulting level of extremeness, guidance is formulated around an explicit target trajectory and adapted to drive the remaining gap toward zero. Guidance strength is therefore treated as the means of reaching a specified scenario rather than as the scenario definition itself.
3. **Evaluate.** We develop a set of diagnostics for both guidance behavior and weather plausibility. Beyond measuring whether the target trajectory is reached, we evaluate how the generated scenarios behave across space, pressure levels, variables, and forecast time. In particular, we construct a historical benchmark in which observed weather states provide plausibility anchors and the corresponding unguided forecast provides a baseline for quantifying the effects of guidance.

If successful, such a framework enables several forms of controlled extreme scenario generation. Starting from a given weather state, one may generate alternative future trajectories at different prescribed levels of extremeness. Data-driven extreme trajectories may instead be used as explicit targets for constructing weather storylines conditioned on a particular initial state. In either case, multiple trajectories and intensity levels can be sampled to explore not a single deterministic extreme, but a family of guided scenarios. Although extreme events provide the focus of this thesis, the underlying idea is more general: guidance can be used to

explore alternative weather trajectories that deliberately depart from the base forecast toward any suitably defined target.

The remainder of the thesis is organized as follows. Chapter 2 introduces the technical background on flow matching and guidance, reviews existing approaches to controlled weather generation, and describes ArchesWeather. Chapters 3 to 5 develop the three stages of the framework in turn: specifying the target trajectory, designing the guidance method, and defining the evaluation framework. Chapter 6 presents the experimental results, which are discussed in Chapter 7. Finally, Chapter 8 summarizes the main findings and outlines directions for future work.

Chapter 2

Background

This chapter introduces the theoretical and methodological background required for the development of our guidance approach.

We begin with flow matching, the generative framework underlying ArchesWeather, and introduce the probability paths, noisy states, velocity fields, and clean estimates used throughout this work. We then formulate guidance from a posterior-sampling perspective, derive its flow-matching formulation, and discuss different approaches to guidance strength and control. Next, we review existing approaches to the controlled generation of weather scenarios and position our work within this emerging literature. Finally, we introduce ArchesWeather, the probabilistic weather model used in this thesis, and describe the components relevant to our method, including its deterministic forecast, residual flow model, and sampling procedure.

2.1 Flow matching

Generative modeling frames the task of generating objects $x_1 \in \mathbb{R}^d$ as sampling from a data distribution p_1. Flow matching [16] models p_1 as the endpoint of a continuous probability path

$$(p_t)_{t\in[0,1]},$$

which transports a tractable source distribution p_0, typically a standard Gaussian, to the data distribution p_1.

At flow time t, we denote the current state along this path by z_t, with

$$z_0 \sim p_0, \qquad z_t \sim p_t, \qquad z_1 = x_1 \sim p_1.$$

Throughout this work, we refer to z_0 as the *noise*, intermediate states z_t, $t \in (0,1)$, as *noisy states*, and the endpoint $z_1 = x_1$ as the *clean state*.

The evolution of z_t is governed by a time-dependent velocity field u_t, defining the ordinary differential equation

$$\frac{\mathrm{d}}{\mathrm{d}t} z_t = u_t(z_t). \tag{2.1}$$

Starting from $z_0 \sim p_0$ and integrating this ODE from $t = 0$ to $t = 1$ therefore transforms samples from the source distribution into samples from the data distribution.

2.1.1 Conditional probability paths

Directly specifying the marginal probability path p_t and its corresponding velocity field u_t is generally intractable. Flow matching circumvents this problem by defining a tractable *conditional probability path*

$$p_t(\cdot \mid x_1)$$

for each data sample $x_1 \sim p_1$.

A common choice is the Gaussian conditional probability path

$$p_t(\cdot \mid x_1) = \mathcal{N}\left(\alpha_t x_1,\, \beta_t^2 I\right), \tag{2.2}$$

where the schedules $\alpha_t, \beta_t \in [0,1]$ interpolate between noise and data according to

$$\alpha_0 = 0, \qquad \beta_0 = 1, \qquad \alpha_1 = 1, \qquad \beta_1 = 0.$$

Consequently,

$$p_0(\cdot \mid x_1) = \mathcal{N}(0, I), \tag{2.3}$$

$$p_1(\cdot \mid x_1) = \delta_{x_1}. \tag{2.4}$$

Sampling from the conditional path is equivalent to interpolating a data sample $x_1 \sim p_1$ with independent Gaussian noise $\varepsilon \sim \mathcal{N}(0, I)$:

$$z_t = \alpha_t x_1 + \beta_t \varepsilon \sim p_t(\cdot \mid x_1). \tag{2.5}$$

Marginalizing over the data distribution gives the corresponding marginal probability path

$$p_t(z_t) = \int p_t(z_t \mid x_1)\, p_1(x_1)\, \mathrm{d}x_1. \tag{2.6}$$

2.1.2 Conditional and marginal velocity fields

Each conditional probability path $p_t(\cdot \mid x_1)$ is generated by a corresponding conditional velocity field $u_t(z_t \mid x_1)$.

For the Gaussian path in Equation (2.2), differentiating the interpolation in Equation (2.5) gives

$$u_t(z_t \mid x_1) = \dot{\alpha}_t x_1 + \dot{\beta}_t \varepsilon. \tag{2.7}$$

Substituting

$$\varepsilon = \frac{z_t - \alpha_t x_1}{\beta_t}$$

yields

$$u_t(z_t \mid x_1) = \frac{\dot{\beta}_t}{\beta_t} z_t + \left(\dot{\alpha}_t - \frac{\dot{\beta}_t}{\beta_t} \alpha_t \right) x_1. \tag{2.8}$$

The marginal velocity field generating p_t can then be expressed as the conditional velocity averaged over all clean samples x_1 compatible with the current noisy state z_t:

$$u_t(z_t) = \mathbb{E}_{x_1 \sim p_t(\cdot \mid z_t)} \left[u_t(z_t \mid x_1) \right]. \tag{2.9}$$

While this marginal velocity is generally not available in closed form, the conditional velocity in Equation (2.8) is tractable during training, since both x_1 and the sampled noise ε are known.

2.1.3 Flow-matching objective

Flow matching exploits the fact that a neural network can be trained on the tractable conditional velocity field while learning the corresponding marginal velocity field [16]. Writing $u_\theta(z_t, t)$ for the learned velocity field, the conditional flow-matching objective is

$$\mathcal{L}_{\mathrm{FM}}(\theta) = \mathbb{E}_{\substack{t \sim \mathcal{U}[0,1], \\ x_1 \sim p_1, \\ z_t \sim p_t(\cdot \mid x_1)}} \left[\| u_\theta(z_t, t) - u_t(z_t \mid x_1) \|_2^2 \right]. \tag{2.10}$$

This conditional objective has the same gradient with respect to the model parameters as the corresponding, but intractable, marginal flow-matching objective [16].

After training, $u_\theta(z_t, t)$ approximates the marginal velocity field $u_t(z_t)$. Generation then consists of sampling

$$z_0 \sim p_0$$

and numerically integrating

$$\frac{\mathrm{d}}{\mathrm{d}t} z_t = u_\theta(z_t, t) \tag{2.11}$$

from $t = 0$ to $t = 1$, such that the resulting endpoint z_1 approximately follows p_1.

2.1.4 Linear probability path

A particularly simple choice is the linear, or optimal-transport, probability path

$$z_t = t x_1 + (1 - t)\varepsilon, \qquad \varepsilon \sim \mathcal{N}(0, I), \tag{2.12}$$

corresponding to

$$\alpha_t = t, \qquad \beta_t = 1 - t.$$

Its conditional velocity reduces to the constant displacement

$$u_t(z_t \mid x_1) = x_1 - \varepsilon. \tag{2.13}$$

Training therefore amounts to least-squares regression onto the displacement between the source noise and the clean sample.

At an intermediate flow time $t < 1$, the noisy state z_t does not uniquely identify the clean sample x_1 from which it originates. Instead, the possible clean samples are described by the conditional distribution

$$p_t(x_1 \mid z_t).$$

In practice, a generative model can provide a tractable clean estimate

$$\hat{x}_t := \hat{x}_\theta(z_t, t) \tag{2.14}$$

from the current noisy state. The distinction between the full conditional distribution $p_t(x_1 \mid z_t)$ and the point estimate $\hat{x}_t$ will become important when introducing guidance in the following section.

2.2 Guidance

Without additional guidance, a generative model samples from its base distribution $p_1(x_1)$. In many applications, however, we are interested in generating samples that satisfy an additional desired condition y. From a posterior-sampling perspective, guidance aims to modify the sampling process toward the corresponding conditional distribution

$$p_1(x_1 \mid y).$$

We first formulate guidance from this posterior-sampling perspective and translate the resulting score modification to the flow-matching velocity field. We then introduce a general guidance-strength schedule and review different strategies for choosing its magnitude over the generative trajectory. Finally, we discuss a complementary control perspective, in which guided generation is formulated as an optimization problem over variables that influence the complete generative trajectory.

2.2.1 Guidance as posterior sampling

Rather than considering only the conditional distribution at the clean endpoint, we can define the corresponding distribution over noisy states at any flow time t. By Bayes' rule,

$$p_t(z_t \mid y) \propto p_t(z_t)\, p_t(y \mid z_t). \tag{2.15}$$

Taking the gradient with respect to the noisy state gives the conditional score

$$\nabla_{z_t} \log p_t(z_t \mid y) = \nabla_{z_t} \log p_t(z_t) + \nabla_{z_t} \log p_t(y \mid z_t). \tag{2.16}$$

The first term corresponds to the unguided generative process, while the second introduces the desired condition y. Guidance can therefore be interpreted as modifying the original generation process through the likelihood gradient

$$\nabla_{z_t} \log p_t(y \mid z_t).$$

In general, however, the intermediate likelihood $p_t(y \mid z_t)$ is not directly available. We therefore marginalize over the clean sample x_1 and assume that the condition y depends on the

generative process only through x_1:

$$p_t(y \mid z_t) = \int p_t(y \mid x_1, z_t)\, p_t(x_1 \mid z_t)\, \mathrm{d}x_1 \tag{2.17}$$

$$= \int p(y \mid x_1)\, p_t(x_1 \mid z_t)\, \mathrm{d}x_1, \qquad y \perp z_t \mid x_1 \tag{2.18}$$

$$= \mathbb{E}_{x_1 \sim p_t(\cdot \mid z_t)} \left[p(y \mid x_1) \right] \tag{2.19}$$

$$\approx \int p(y \mid x_1)\, \delta_{\hat{x}_t}(x_1)\, \mathrm{d}x_1, \qquad p_t(x_1 \mid z_t) \approx \delta_{\hat{x}_t}(x_1) \tag{2.20}$$

$$= p(y \mid \hat{x}_t). \tag{2.21}$$

Here, $\hat{x}_t = \hat{x}_\theta(z_t, t)$ denotes the clean estimate obtained from the current noisy state. The approximation replaces the full conditional distribution $p_t(x_1 \mid z_t)$ by a point mass at $\hat{x}_t$, yielding

$$p_t(y \mid z_t) \approx p(y \mid \hat{x}_t). \tag{2.22}$$

This clean-estimate approximation also underlies Diffusion Posterior Sampling [5].

The likelihood $p(y \mid x_1)$ can in principle be modeled in different ways. A convenient choice for gradient-based guidance is to define it through a differentiable loss $\mathcal{L}$:

$$p(y \mid x_1) \propto \exp\left(-\mathcal{L}(x_1, y)\right). \tag{2.23}$$

It follows that

$$\nabla_{z_t} \log p(y \mid \hat{x}_t) = -\nabla_{z_t} \mathcal{L}(\hat{x}_t, y), \tag{2.24}$$

where the gradient is propagated through the clean estimate $\hat{x}_t = \hat{x}_\theta(z_t, t)$.

2.2.2 From scores to flow-matching velocities

The previous derivation expresses guidance in terms of scores. To apply it to flow matching, we require the corresponding modification of the velocity field.

For the Gaussian probability path introduced in Equation (2.2), the marginal velocity field and score are related by

$$u_t(z_t) = a_t \nabla_{z_t} \log p_t(z_t) + b_t z_t, \tag{2.25}$$

where

$$a_t := \beta_t^2 \left(\frac{\dot{\alpha}_t}{\alpha_t} - \frac{\dot{\beta}_t}{\beta_t} \right), \tag{2.26}$$

$$b_t := \frac{\dot{\alpha}_t}{\alpha_t}. \tag{2.27}$$

Applying the same relation to the conditional distribution gives

$$u_t(z_t \mid y) = a_t \nabla_{z_t} \log p_t(z_t \mid y) + b_t z_t. \tag{2.28}$$

Substituting the conditional-score decomposition yields

$$u_t(z_t \mid y) = a_t \left[\nabla_{z_t} \log p_t(z_t) + \nabla_{z_t} \log p_t(y \mid z_t) \right] + b_t z_t \tag{2.29}$$

$$= u_t(z_t) + a_t \nabla_{z_t} \log p_t(y \mid z_t). \tag{2.30}$$

Using the clean-estimate approximation derived above, we obtain

$$u_t(z_t \mid y) \approx u_t(z_t) + a_t \nabla_{z_t} \log p(y \mid \hat{x}_t). \tag{2.31}$$

Under the loss-based likelihood in Equation (2.23), this can equivalently be written as

$$u_t(z_t \mid y) \approx u_t(z_t) - a_t \nabla_{z_t} \mathcal{L}(\hat{x}_t, y). \tag{2.32}$$

Thus, gradient-based guidance in flow matching consists of evaluating a loss on the clean estimate $\hat{x}_t$, differentiating it with respect to the current noisy state z_t, and using the resulting gradient to modify the velocity field.

2.2.3 Guidance-strength schedules

The coefficient a_t in Equation (2.32) is determined by the underlying probability path. Using this coefficient preserves the posterior-sampling interpretation, up to the clean-estimate approximation introduced above.

In practice, guidance methods often rescale this contribution to obtain stronger or weaker control over the generated samples. We therefore replace a_t by a general guidance-strength schedule

$$\lambda_t := w \, \gamma_t \, \nu_t, \tag{2.33}$$

where

- $w \geq 0$ controls the overall guidance strength;
- $\gamma_t \geq 0$ controls how guidance varies along the flow;
- $\nu_t > 0$ is an optional algorithm-specific normalization factor.

We then define the *guided velocity field* as

$$\tilde{u}_t(z_t \mid y) := u_t(z_t) + \lambda_t \nabla_{z_t} \log p(y \mid \hat{x}_t) \tag{2.34}$$

$$= u_t(z_t) - \lambda_t \nabla_{z_t} \mathcal{L}(\hat{x}_t, y). \tag{2.35}$$

We refer to $u_t(z_t)$ as the *unguided velocity field*.

Setting

$$\lambda_t = a_t$$

recovers the approximate conditional velocity field in Equation (2.31). For a general choice of λ_t, however, the resulting dynamics no longer necessarily correspond to sampling from $p_1(x_1 \mid y)$. Instead, the guidance strength becomes an algorithmic parameter controlling the trade-off between the original generative dynamics and the imposed condition.

The choice of λ_t varies substantially across applications. The simplest approach is to use a constant guidance strength throughout sampling,

$$\lambda_t = w, \tag{2.36}$$

where the scalar $w > 0$ controls the overall strength of guidance and is treated as a tunable hyperparameter. Fixed guidance scales are widely used in classifier-free guidance; for instance, Esser et al. [8] evaluate rectified-flow models using several values of w. A similar strategy is used for weather guidance by Ueyama, Kawamoto, and Kera [26], who tune a single guidance scale throughout the GenCast diffusion process, omitting guidance only at the final noise level. Loss-Guided Diffusion [24] similarly tunes an overall task-level guidance strength, while additionally introducing noise-dependent normalization to control the magnitude of the resulting gradient.

Other approaches prescribe the temporal variation of guidance using the noise schedule of the pretrained generative model. In the original classifier-guidance formulation [7], and subsequently in Universal Guidance [1], the guidance contribution is scaled according to

$$\lambda_t = w\sqrt{1 - \bar{\alpha}_t^{\text{diff}}}, \tag{2.37}$$

where $\bar{\alpha}_t^{\text{diff}}$ denotes the cumulative signal coefficient of the pretrained diffusion process. Consequently, $\sqrt{1-\bar{\alpha}_t^{\text{diff}}}$ is the standard deviation of the noise present at diffusion step t. The overall gain w is again tuned for the particular guidance objective, while the temporal shape of the schedule is inherited directly from the pretrained model's noise schedule rather than chosen independently.

Finally, the guidance strength can be adapted dynamically using quantities observed during sampling. Climate in a Bottle [4], for example, normalizes its classifier-guidance contribution relative to the magnitude of the unguided generative update. Schematically, in the notation used here, this corresponds to

$$\lambda_t \propto w \frac{\|u_t(z_t)\|_2}{\|\nabla_{z_t}\mathcal{L}(\hat{x}_t, y)\|_2}, \tag{2.38}$$

such that the guidance vector remains a fixed fraction of the unguided generative update. From an optimization perspective, Shen et al. [22] similarly interpret guidance strength as a step size and propose a Polyak-style adaptive rule. Their update scales the guidance gradient using both its own norm and the magnitude of the unguided model prediction, adapting the effective step size at each sampling step rather than prescribing it beforehand.

These examples illustrate that, even within the same local guidance construction, the guidance schedule is not uniquely determined in practice. Existing approaches range from tuning a constant global gain w, to combining w with a temporal schedule inherited from the pretrained model, to adapting the effective guidance magnitude dynamically during sampling.

2.2.4 Guidance as control

A complementary perspective formulates guided generation as a control problem over the generative trajectory. Rather than constructing a local approximation of a conditional score, the objective is to modify variables that influence the trajectory such that its endpoint satisfies a desired condition.

For consistency, we express the following methods using the notation introduced in this chapter rather than that of the original papers. We denote the generative state by z_t, the frozen pretrained velocity field by $u_t(z_t)$, and the terminal guidance objective by $\mathcal{L}(z_1, y)$.

FlowGrad [17] introduces an additive control g_t into the pretrained dynamics,

$$\frac{\mathrm{d}}{\mathrm{d}t} z_t = u_t(z_t) + g_t, \tag{2.39}$$

and optimizes the complete control trajectory according to

$$\min_{\{g_t\}} \mathcal{L}(z_1, y) + \kappa \int_0^1 \|g_t\|_2^2 \, \mathrm{d}t, \tag{2.40}$$

where g_t determines how the pretrained dynamics are modified at flow time t, and $\kappa > 0$ penalizes large deviations from the original flow. Since the terminal state z_1 depends on all preceding controls, their gradients are obtained by backpropagating the terminal objective through the ODE trajectory.

D-Flow [2] instead leaves the velocity field unchanged and optimizes only the source noise:

$$\min_{z_0} \mathcal{L}\big(z_1(z_0), y\big), \qquad \frac{\mathrm{d}}{\mathrm{d}t} z_t = u_t(z_t). \tag{2.41}$$

Here, $z_1(z_0)$ emphasizes that the generated endpoint is determined by the initial noise through integration of the frozen flow. The terminal gradient is therefore propagated through the complete generative trajectory back to z_0.

OC-Flow [27] places such approaches within a general optimal-control framework. Using the same notation, its Euclidean formulation can be viewed as optimizing a control trajectory

$$\min_{\{g_t\}} \mathcal{L}(z_1, y) + \int_0^1 \mathcal{C}(g_t) \, \mathrm{d}t, \qquad \frac{\mathrm{d}}{\mathrm{d}t} z_t = u_t(z_t) + g_t, \tag{2.42}$$

where $\mathcal{C}$ is a running cost that penalizes the applied control. OC-Flow derives the control updates using optimal-control theory and shows that existing backpropagation-through-ODE approaches can be interpreted within this framework.

The posterior-sampling and control perspectives therefore provide two complementary views of guided generation. Posterior-style methods construct local corrections to the generative dynamics from a conditioning objective, while control-based methods formulate guidance as an optimization problem over variables that influence the complete generative trajectory.

These perspectives provide the conceptual basis for the weather-specific approaches to controlled generation reviewed next.

2.3 Guided generation of weather scenarios

Having introduced the posterior-sampling and control perspectives on guidance, we now turn to their application in weather generation. Guided generation in weather and climate models

remains comparatively recent, but existing work already spans several settings, from controlling individual atmospheric states to modifying complete weather trajectories. The approaches differ primarily in where control enters the generation process and how the desired event is specified.

For multi-step weather trajectories, we use $n = 0, \dots, N$ to index weather time, while t continues to denote the internal diffusion or flow time of the generative process. Sampling-time conditioning has also been explored beyond extreme-event generation, for example by incorporating external forecast information during diffusion sampling [13]. Here, we focus specifically on methods aimed at controlling rare or extreme weather outcomes.

State-level guidance. At the most local level, guidance can be used to control the properties of a single generated atmospheric state. A representative example is Climate in a Bottle (cBottle) [4], which uses classifier guidance for the on-demand generation of extreme atmospheric states. Unlike autoregressive weather models, cBottle generates independent multivariate states conditioned on slowly varying quantities such as sea-surface temperature and solar position.

To generate tropical cyclones at user-specified locations, the authors train a classifier on historical cyclone observations and use its gradient during diffusion sampling. Expressed schematically in the notation introduced above, the guided dynamics take the form

$$\tilde{u}_t(z_t \mid y) = u_t(z_t) - \lambda_t \nabla_{z_t} \mathcal{L}_{\mathrm{TC}}(z_t, y), \tag{2.43}$$

where y specifies the desired tropical-cyclone locations and $\mathcal{L}_{\mathrm{TC}}$ denotes the classifier loss. The guidance strength is adaptively normalized relative to the unguided generative update, as discussed in Section 2.2.3. The resulting samples contain cyclones at the requested locations while retaining correlated structure across atmospheric variables [4].

Since cBottle generates atmospheric states independently, the method controls the occurrence and location of an event in a single state rather than its temporal evolution.

Forecast guidance. Guidance has also been applied directly within probabilistic weather forecasting. Ueyama, Kawamoto, and Kera [26] use gradient-based guidance during GenCast diffusion sampling to reduce precipitation over a prescribed target region. In contrast to approaches that perturb an atmospheric state before forecasting, the intervention is applied within the generative sampling process itself.

Using the guidance notation introduced above, the mechanism can be represented schemati-

cally as

$$\tilde{u}_t(z_t \mid y) = u_t(z_t) - \lambda \nabla_{z_t} \mathcal{L}_{\text{precip}}(\hat{x}_t, y), \tag{2.44}$$

where $\mathcal{L}_{\text{precip}}$ measures the precipitation-reduction objective on the current clean estimate $\hat{x}_t$. The gradient is propagated through the clean estimate to the noisy state z_t and used to modify the generative update.

The guidance scale λ is fixed during sampling and controls the strength of the intervention. Increasing it produces stronger precipitation reduction, but also larger deviations from the original forecast. Accordingly, the authors evaluate not only whether the intervention succeeds, but also its effects on the surrounding atmospheric state through variable-wise and vertical perturbation structures, deviations of the generative trajectory, and cross-model transferability [26].

Trajectory-level control. Control objectives can also be defined over complete weather trajectories. A particularly relevant example is Whittaker and Di Luca [28], who construct extreme heatwave storylines by exploiting the differentiability of NeuralGCM. Starting from a historical atmospheric state x_0, they optimize a small perturbation δx_0 to the initial condition whose subsequent evolution produces a more extreme heatwave:

$$\delta x_0^\star = \arg\min_{\delta x_0} \left[\mathcal{L}_{\text{HW}}\big(F_{0:N}(x_0 + \delta x_0)\big) + \kappa \|\delta x_0\|_2^2 \right], \tag{2.45}$$

where $F_{0:N}$ denotes the differentiable NeuralGCM rollout, $\mathcal{L}_{\text{HW}}$ penalizes trajectories that fail to attain the desired heatwave intensity, and $\kappa > 0$ controls the penalty on the magnitude of the initial perturbation.

Gradients are propagated through the complete model trajectory back to the initial atmospheric state. Applied to the 2021 Pacific Northwest heatwave, the optimization produces trajectories more extreme than those found in a conventional ensemble while retaining characteristic large-scale structures of the event [28].

This construction differs from the generative-guidance approaches above. NeuralGCM is used as a differentiable dynamical model, and control is achieved by optimizing the initial atmospheric condition rather than by intervening within a generative sampling process. It therefore provides an example of trajectory-level control through initial-condition optimization.

Other approaches control trajectories without differentiating an event loss through the forecast model. He and Guo [9] use Intensity–Duration–Frequency profiles as likelihood constraints

during sequential generation, preferentially retaining trajectories compatible with the desired extreme profile. Related rare-event methods instead clone or prune trajectories according to their likelihood of developing into the target event [15].

A further alternative is to incorporate the desired condition during training rather than impose it on a pretrained generator at inference time. Loer, Schillinger, and Sippel [18], for example, learn a conditional generative distribution of temperature fields given atmospheric circulation and global warming level. In the Bayesian terminology introduced above, this corresponds to learning $p(x \mid y)$ directly, rather than starting from a pretrained model of $p(x)$ and introducing the condition through $p(y \mid x)$ during sampling.

Taken together, these studies illustrate several routes to controlled weather generation: guiding individual atmospheric states, intervening during probabilistic forecast generation, optimizing initial conditions for complete trajectories, selectively sampling rare trajectories, and learning conditional generators directly.

2.4 ArchesWeather

We investigate this problem using ArchesWeather [6], a probabilistic medium-range weather forecasting model that combines a deterministic forecast with a flow-matching model of the forecast residual. Having established the general flow-matching and guidance formulations above, we now describe the components of ArchesWeather required for the development of our method and refer to the original work for architectural and training details.

2.4.1 Dataset

ArchesWeather is trained on the ERA5 reanalysis dataset [11], using one of the versions preprocessed by WeatherBench 2 [21]. The data are represented on a regular 1.5° latitude–longitude grid with 121×240 spatial points.

Each weather state

$$x_n \in \mathbb{R}^{82\times121\times240}$$

contains 82 atmospheric fields. Four are surface variables: 2 m temperature, mean sea-level pressure, and the two components (u and v) of the 10 m wind. The remaining 78 fields correspond to six upper-air variables

$$\{\text{geopotential}, \text{temperature}, \text{specific humidity}, u, v, \text{vertical velocity}\}$$

evaluated at the 13 pressure levels

$$\{50, 100, 150, 200, 250, 300, 400, 500, 600, 700, 850, 925, 1000\} \text{ hPa}.$$

Weather states are available at 00:00, 06:00, 12:00, and 18:00 UTC. Following the WeatherBench 2 evaluation protocol, dates from January 2020 onward are reserved for evaluation.

Both the deterministic and generative components operate in normalized data space. Unless stated otherwise, the variables introduced below therefore refer to their normalized representation.

2.4.2 Forecasting setup

ArchesWeather generates forecast trajectories autoregressively. Following the convention introduced above, we index forecast lead time by n, which we refer to as *weather time*. At each weather step, the model produces a forecast for the next state x_{n+1}.

Rather than generating x_{n+1} directly, ArchesWeather decomposes the forecast into a deterministic component and a stochastic residual component. The deterministic model first produces a base forecast

$$\hat{x}^{\text{det}}_{n+1}.$$

The corresponding ground-truth residual is defined as

$$r_{n+1} := x_{n+1} - \hat{x}^{\text{det}}_{n+1}. \tag{2.46}$$

A conditional flow-matching model is then trained to generate this residual. At inference time, a sampled residual $\hat{r}_{n+1}$ is combined with the deterministic prediction as

$$\hat{x}_{n+1} = \hat{x}^{\text{det}}_{n+1} + \hat{r}_{n+1}. \tag{2.47}$$

Sampling different residuals while keeping the deterministic forecast fixed therefore produces different members of an ensemble forecast.

2.4.3 Residual flow model

Generating each stochastic residual requires a complete flow-matching trajectory. ArchesWeather therefore involves two nested notions of time: the weather-time index n, which

advances the autoregressive forecast, and the flow time t, which describes the generation of a residual within each weather step.

Following the notation introduced in the previous sections, the clean endpoint of the flow is now the forecast residual r_{n+1}, while z_t denotes its noisy state at flow time t.

ArchesWeather parameterizes the Gaussian probability path in terms of a noise level s_t, which decreases linearly from 1 at the beginning of the flow to 0 at its endpoint. The noisy residual is constructed as

$$z_t = (1 - s_t)\, r_{n+1} + s_t\, \varepsilon, \qquad \varepsilon \sim \mathcal{N}(0, I). \tag{2.48}$$

This corresponds directly to the Gaussian probability path introduced in Equation (2.2), with

$$\alpha_t = 1 - s_t, \qquad \beta_t = s_t. \tag{2.49}$$

The flow therefore starts from Gaussian noise and progressively approaches the clean residual r_{n+1}.

Unlike the velocity-field parameterization introduced in the generic formulation above, the ArchesWeather residual model is trained to predict the clean residual from the noisy state. We denote this estimate by

$$\hat{r}_t := r_\theta(z_t, t, c_n), \tag{2.50}$$

where

$$c_n = \left(x_{n-1}, x_n, \hat{x}^{\mathrm{det}}_{n+1}\right)$$

collects the conditioning information provided to the residual model.

The model is trained using the clean-prediction objective

$$\mathcal{L}_{\mathrm{Arches}}(\theta) = \mathbb{E}_{t, r_{n+1}, \varepsilon}\left[\left\| r_\theta(z_t, t, c_n) - r_{n+1} \right\|_2^2\right]. \tag{2.51}$$

In the notation of the previous sections, $\hat{r}_t$ thus plays the role of the clean estimate $\hat{x}_t$. Since the flow generates a residual rather than the complete weather state, the corresponding clean weather estimate at flow time t is

$$\hat{x}_t := \hat{x}^{\mathrm{det}}_{n+1} + \hat{r}_t. \tag{2.52}$$

This quantity will later provide the state on which our guidance objectives are evaluated.

2.4.4 Sampling

Although the network predicts the clean residual, sampling requires the corresponding velocity field. From Equation (2.48), the noise associated with the clean estimate $\hat{r}_t$ is

$$\hat{\varepsilon}_t = \frac{z_t - (1 - s_t)\hat{r}_t}{s_t}. \tag{2.53}$$

For the linear path

$$\alpha_t = 1 - s_t, \qquad \beta_t = s_t,$$

the velocity is the displacement between the clean residual estimate and the corresponding noise estimate:

$$\begin{aligned} u_\theta(z_t, t, c_n) &= \hat{r}_t - \hat{\varepsilon}_t && (2.54)\\ &= \hat{r}_t - \frac{z_t - (1 - s_t)\hat{r}_t}{s_t} && (2.55)\\ &= \frac{\hat{r}_t - z_t}{s_t}. && (2.56) \end{aligned}$$

ArchesWeather discretizes the flow into $T = 25$ sampling steps. In the following, $t = 0, \ldots, T$ denotes the corresponding discrete flow-step index. Starting from Gaussian noise

$$z_0 \sim \mathcal{N}(0, I),$$

the residual is generated through Euler integration,

$$z_{t+1} = z_t + h_t\, u_\theta(z_t, t, c_n), \qquad h_t = s_t - s_{t+1}, \qquad t = 0, \ldots, T-1. \tag{2.57}$$

Since the noise level decreases from $s_0 = 1$ to $s_T = 0$, the sequence $\{z_t\}_{t=0}^{T}$ progressively evolves from noise toward a clean residual.

After the final flow step, the endpoint is taken as the sampled residual,

$$\hat{r}_{n+1} := z_T, \tag{2.58}$$

which is added to the deterministic forecast to obtain

$$\hat{x}_{n+1} = \hat{x}^{\text{det}}_{n+1} + \hat{r}_{n+1} = \hat{x}^{\text{det}}_{n+1} + z_T. \tag{2.59}$$

Figure 2.1 summarizes the computations performed within a single weather step. This structure will be central to the implementation and evaluation of our guidance method.

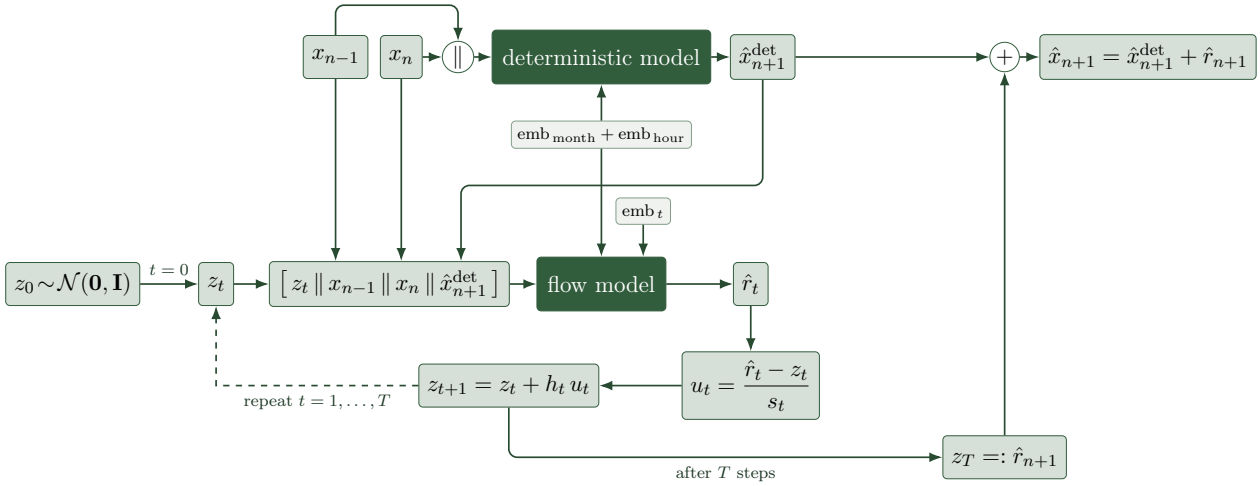


Figure 2.1 – Computational graph of one ArchesWeather forecast step. The deterministic model produces the base forecast $\hat{x}^{\text{det}}_{n+1}$, while the flow model generates the residual through T Euler steps before both components are combined.

The nested residual-flow structure of ArchesWeather provides the concrete generative process on which our guidance method will operate. Before developing that method in Chapter 4, we first specify the weather trajectories toward which the model should be steered in Chapter 3.

Chapter 3

Specifying events of interest

The first component of our framework is to specify the event toward which the generative model should be guided. In this thesis, we represent such an event through a scalar target trajectory

$$y^\star_{1:N} = (y^\star_1, \dots, y^\star_N)$$

over N weather steps, together with a mask m that determines which quantity of the forecast state is being targeted. Here, $n = 1, \dots, N$ indexes weather time, while t remains reserved for the internal flow time of the generative model.

Throughout this thesis, we focus on regional surface-temperature heatwaves. Accordingly, the mask selects surface temperature over a prescribed spatial region, and $y^\star_{1:N}$ describes the desired evolution of its regional average. The broader framework is not restricted to this particular choice: different differentiable functionals of the weather state could be used to specify other types of target events. The particular mask and target construction developed here should therefore be viewed as one instantiation of the more general *Specify–Guide–Evaluate* framework.

We distinguish two ways of constructing target trajectories. For evaluating the guidance method itself, we use simple controlled targets defined relative to a matched unguided forecast. This allows us to systematically vary target intensity and study how guidance behaves as the requested deviation increases. For the final storyline generation, we instead construct targets from historically realized trajectories and select those satisfying a desired extremeness criterion. In both cases, the guidance method receives the same object: a prescribed trajectory $y^\star_{1:N}$ for the masked quantity.

3.1 Target variable and region

We jointly specify the target variable and spatial region through a mask over the weather state. Let

$$x_n \in \mathbb{R}^{V \times I \times J}$$

denote a weather state, where V indexes atmospheric fields and $I = 121$, $J = 240$ index latitude and longitude. We denote the corresponding coordinates by

$$(\varphi_i, \psi_j), \qquad i = 1, \dots, I, \quad j = 1, \dots, J.$$

Let $v^\star$ denote the target variable and let

$$k_{ij} \in \{0, 1\}$$

indicate whether spatial grid cell (i, j) lies inside the selected region. In this work, the region is defined as a rectangular bounding box on the latitude–longitude grid.

The mask is non-zero only for the selected variable and inside the selected region. To obtain a true area-weighted regional average, each included grid cell is weighted according to the surface area it represents. For a regular latitude–longitude grid, the area contribution of latitude row i is proportional to

$$a_i = \int_{\beta_i^-}^{\beta_i^+} \cos\varphi \, \mathrm{d}\varphi, \tag{3.1}$$

where $[\beta_i^-, \beta_i^+]$ denotes the latitude extent of the grid cell. The constant longitudinal cell width cancels during normalization.

We therefore define the mask as

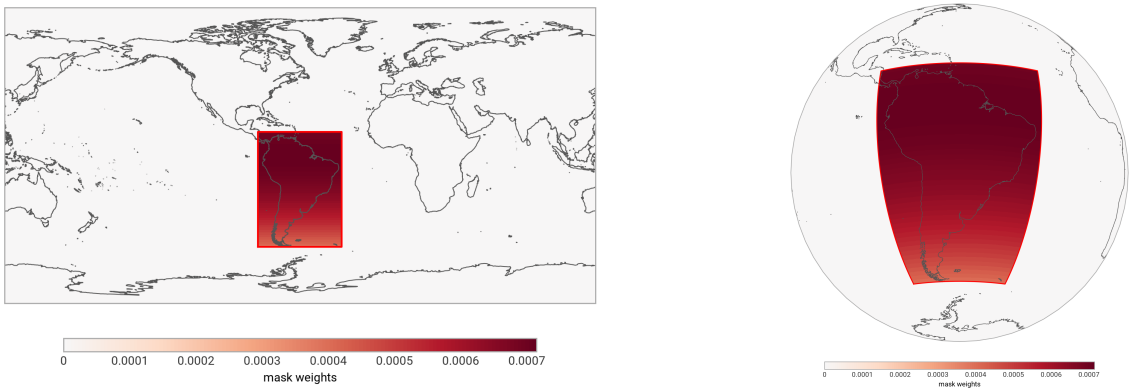
$$m_{vij} = \frac{\mathbf{1}[v = v^\star] \, k_{ij} \, a_i}{\sum_{i',j'} k_{i'j'} \, a_{i'}}, \tag{3.2}$$

such that

$$\sum_{v,i,j} m_{vij} = 1.$$

The corresponding masked-average operator is

$$\mathcal{M}_m(x_n) := \sum_{v,i,j} m_{vij} \, x_{n,vij}. \tag{3.3}$$



(a) Data-space representation.

(b) Projected representation.

Figure 3.1 – Example of the spatial target mask. The rectangular support determines which grid cells are included, while the latitude-dependent weights account for differences in physical grid-cell area.

For our experiments, $\mathcal{M}_m(x_n)$ therefore corresponds to the area-weighted mean surface temperature inside the selected region.

Area weighting prevents the regional average from being biased toward high-latitude grid cells, which represent a smaller physical area on the sphere. This follows the latitude-weighting convention used in WeatherBench 2 [21].

Figure 3.1 illustrates the resulting area-weighted mask for an example target region over South America. The rectangular support is constant in longitude, while the normalized mask weights vary with latitude according to the physical area represented by each grid cell.

Using a binary spatial support also makes the target region explicit. Within the selected region, cells differ only through their physical area and are otherwise treated uniformly. This avoids introducing an additional spatial preference, as would occur, for example, with a bell-shaped mask whose center is weighted more strongly than its boundaries. Normalizing the mask to sum to one further ensures that $\mathcal{M}_m$ remains an average and is therefore directly comparable across regions of different sizes.

Although m jointly specifies one particular variable and region, its spatial support k_{ij} can also be reused to construct analogous averages for other atmospheric fields. This becomes useful later when evaluating how guidance applied to surface temperature affects other variables and pressure levels over the same region.

3.2 Target trajectory

Once the mask has been specified, the remaining task is to define the desired evolution of the masked quantity,

$$y^{\star}_{1:N}.$$

The appropriate construction depends on the purpose of the experiment. We therefore separate the controlled targets used to evaluate the guidance method from the data-driven targets used for extreme-event storylines.

3.2.1 Controlled targets for method evaluation

TODO: explain here that we compute the percentage wrt to the ensemble average to retain the ensemble distribution over time. Otherwise all members collapse to a single trajectory.

When evaluating the guidance algorithm, our primary objective is to understand how it behaves as the requested deviation from the original forecast becomes increasingly demanding. For this purpose, we deliberately use a simple and interpretable target construction.

Each guided rollout is paired with a corresponding unguided rollout

$$\{x^{\mathrm{ung}}_n\}_{n=1}^{N}.$$

The two rollouts use the same initial atmospheric context and matched random noise realizations throughout the autoregressive forecast. Their difference therefore isolates, as far as possible, the effect introduced by guidance rather than differences arising from stochastic sampling.

We prescribe the target trajectory as a relative increase over the masked average of this matched unguided rollout. Let ρ_N denote the maximum requested relative increase at the final weather step. We linearly increase the target intensity over the forecast horizon according to

$$\rho_n := \frac{n}{N}\rho_N, \qquad n = 1, \ldots, N. \tag{3.4}$$

The target trajectory is then defined as

$$y^{\star}_n := (1 + \rho_n)\, \mathcal{M}_m(x^{\mathrm{ung}}_n). \tag{3.5}$$

All target values are defined after transforming the relevant field back to physical units. For

temperature, a multiplicative relative change depends on the chosen temperature scale and therefore does not constitute a scale-independent measure of physical extremeness. Accordingly, ρ_N is used here purely as a controlled intensity parameter for systematically stress-testing the guidance method. It should not be interpreted as a climatological definition of heatwave extremeness.

Given a guided rollout x_n^{gui}, the remaining target gap at weather step n is

$$\xi_n := \mathcal{M}_m(x_n^{\text{gui}}) - y_n^\star = \mathcal{M}_m(x_n^{\text{gui}}) - (1+\rho_n)\mathcal{M}_m(x_n^{\text{ung}}). \tag{3.6}$$

This formulation separates the desired outcome from the parameters used by the guidance algorithm to achieve it. Rather than increasing a guidance-strength parameter and observing how extreme the resulting forecast becomes, we first prescribe the desired trajectory and subsequently ask which guidance behavior is required to reach it. Guidance strength thus becomes a means of attaining a target rather than the definition of the target itself.

This distinction is central to our approach. It allows guidance methods and hyperparameters to be compared at fixed target intensities and makes failure directly interpretable as an inability to close a known target gap.

3.2.2 Data-driven targets for storyline generation

The controlled targets above are useful for evaluating the guidance method, but their relative intensity parameter does not provide a climatological definition of an extreme event. For the generation of final weather storylines, we therefore consider a different target-construction strategy in which the trajectory itself is anchored in historical weather evolution.

Because ArchesWeather conditions its generative forecast on the two preceding weather states, we define the initial atmospheric context as the pair

$$(x_{-1}, x_0).$$

Given this context and the target mask m, we retrieve historical analogue contexts that are sufficiently similar according to a chosen similarity criterion. For each analogue, we then consider the subsequent evolution over the same N-step forecast horizon. The particular similarity metric and analogue-selection procedure are experimental choices and are specified in the corresponding experimental setup.

This produces a collection of historically realized trajectories

$$\left\{\tau_{1:N}^{(k)}\right\}_{k=1}^{K_{\text{hist}}},$$

where

$$\tau_n^{(k)} := \mathcal{M}_m\left(x_n^{(k)}\right) \tag{3.7}$$

denotes the masked trajectory associated with historical analogue k.

These trajectories form an empirical reference set of historically realized evolutions associated with analogous initial situations. From this set, candidate trajectories can be selected according to a desired event property.

More generally, let

$$S(\tau_{1:N}) \tag{3.8}$$

denote an application-specific functional used to characterize a trajectory. For heatwaves, possible choices include the maximum regional temperature anomaly, the mean anomaly over the event, the cumulative anomaly, or another heatwave-specific metric. The precise choice of S determines what aspect of the event is considered extreme, but is not intrinsic to the guidance framework.

A set of storyline candidates can therefore be written abstractly as

$$\mathcal{T}_{\text{ext}} := \{\tau_{1:N} \ : \ S(\tau_{1:N}) \in \mathcal{A}\}, \tag{3.9}$$

where $\mathcal{A}$ specifies the desired range or quantile of extremeness. An entire selected trajectory is then used as the guidance target,

$$y_{1:N}^{\star} := \tau_{1:N}. \tag{3.10}$$

Importantly, selection is performed at the level of complete trajectories rather than independently at each weather step. The temporal evolution of a selected target therefore remains inherited from a coherent candidate trajectory instead of being assembled from unrelated extreme values at individual lead times.

This construction also separates the definition of an extreme event from the guidance mechanism. Different applications may use different similarity criteria, selection procedures, or extremeness functionals while leaving the subsequent guidance problem unchanged. Once m and $y_{1:N}^{\star}$ have been specified, the guidance algorithm only needs to steer the masked forecast toward the prescribed trajectory.

The historical trajectories used to construct the target set also provide natural reference states for the plausibility analysis developed in Chapter 5. Thus, the same historical information used to define the target can later be used to assess whether the guided forecast remains consistent with weather states associated with comparable evolutions.

3.3 Pipeline overview

The specification procedure can now be summarized independently of how the target trajectory is constructed. A guided scenario is specified by

1. an initial atmospheric context (x_{-1}, x_0);
2. a mask m, specifying the target variable and spatial region;
3. a scalar target trajectory $y^{\star}_{1:N}$ for the masked quantity.

The target trajectory may be constructed in different ways. For controlled method evaluation, it is derived from the matched unguided forecast through Equation (3.5). For storyline generation, it is selected from historical analogue trajectories according to an application-specific event criterion.

Once the initial context is fixed, the output of the specification stage is therefore

$$(m,\, y^{\star}_{1:N})\,.$$

These two objects define *what* the model should generate. The next chapter addresses the complementary problem of *how* to modify the generative process in order to realize that target.

Chapter 4

Guidance design

The previous chapter specified the scenario to be generated through an initial atmospheric context, a target mask m, and a target trajectory $y^{\star}_{1:N}$. We now address the second component of the framework: how the generative forecast can be steered towards this prescribed target.

We formulate this as a *target-driven guidance* problem. At each weather step n, the target $y^{\star}_{n}$ is fixed before generation begins. During the internal residual flow, the current clean-state estimate is repeatedly compared with this target, and the resulting discrepancy is used to modify the generative dynamics. The central problem is therefore not to choose a guidance strength and observe the resulting scenario, but rather to determine the guidance required to realize a scenario that has been specified in advance.

We first introduce the common guidance formulation used throughout this chapter. We then develop three strategies for realizing the prescribed target: Close-Gap, which locally controls the remaining target gap; Optimize-Schedule, which jointly optimizes the interventions across the flow; and Optimize-Gain, which fixes the relative flow-time profile and optimizes a single guidance gain. Finally, we introduce Universal Guidance and adversarial weather forecasting as baseline approaches.

4.1 Target-driven guidance

Guidance operates independently at every weather step of the autoregressive forecast. For notational simplicity, we denote by $\hat{x}_{n,t}$ the clean-state estimate of the weather state being generated at weather step n, evaluated at internal flow time t. Thus, n continues to index weather time, while t refers exclusively to the generative flow.

Although the guidance problem is solved separately at each weather step, its effects are not

independent across weather time. Once the guided state x_n^{gui} has been generated, it becomes part of the conditioning context used to generate the following state. Guidance applied at one weather step can therefore propagate through the subsequent autoregressive forecast.

4.1.1 Guidance objective

At weather step n, the target construction introduced in Chapter 3 provides a scalar target value $y_n^\star$. Given the mask m, we measure the discrepancy between the current clean-state estimate and this target by

$$\xi_{n,t} := \mathcal{M}_m(\hat{x}_{n,t}) - y_n^\star, \tag{4.1}$$

where $\mathcal{M}_m$ denotes the masked average defined in Chapter 3. We refer to $\xi_{n,t}$ as the *target gap*.

The corresponding guidance loss is the squared target gap,

$$\mathcal{L}_{n,t} := \xi_{n,t}^2 = \left(\mathcal{M}_m(\hat{x}_{n,t}) - y_n^\star\right)^2. \tag{4.2}$$

This formulation separates the specification of the desired scenario from the mechanism used to realize it. In the controlled experiments of Chapter 3, for example,

$$y_n^\star = (1 + \rho_n)\, \mathcal{M}_m(x_n^{\mathrm{ung}}), \tag{4.3}$$

whereas for historical storyline generation $y_n^\star$ is obtained from a selected historical trajectory. The guidance algorithms developed below are unchanged by this choice: once $y_n^\star$ has been specified, they operate only on the resulting target gap.

After completing the internal flow, the generated weather state is denoted by x_n^{gui}. Its final realized target gap is therefore

$$\xi_n := \mathcal{M}_m(x_n^{\mathrm{gui}}) - y_n^\star = \xi_{n,T}. \tag{4.4}$$

The objective of the guidance methods in this chapter is to reduce $|\xi_n|$, ideally reaching $\xi_n = 0$. Importantly, this does not imply that arbitrary changes to the atmospheric state are considered acceptable. Reaching the prescribed target describes only whether guidance has succeeded with respect to the specified event; the meteorological plausibility of the resulting forecast is evaluated separately.

TODO: say something about alternative objectives and option to define a curved mask or push top k% area.

4.1.2 Gradient-based flow guidance

As introduced in the background chapter, loss-based guidance modifies the generative velocity using the gradient of a differentiable objective with respect to the current noisy state. At weather step n and flow time t, we compute

$$g_{n,t} := \nabla_{z_{n,t}} \mathcal{L}_{n,t}. \tag{4.5}$$

For the squared target-gap loss in Equation (4.2), this becomes

$$g_{n,t} = 2\xi_{n,t} \nabla_{z_{n,t}} \mathcal{M}_m(\hat{x}_{n,t}). \tag{4.6}$$

Although the objective itself is defined only through the masked quantity $\mathcal{M}_m$, the gradient is taken with respect to the complete noisy residual $z_{n,t}$. It therefore propagates through the mapping from the noisy state to the clean-state estimate and can contain non-zero components outside the explicitly targeted variable and region. The intervention is thus not obtained by directly adding the mask to the generated weather field. Instead, the gradient describes the local change of the noisy state that modifies the masked quantity through the learned generative model.

Using the notation introduced in the background chapter, the guided velocity can be written as

$$\tilde{u}_{n,t} = u_{n,t} - \lambda_{n,t} g_{n,t}, \tag{4.7}$$

followed by the usual Euler update

$$z_{n,t+1} = z_{n,t} + h_t \tilde{u}_{n,t}. \tag{4.8}$$

It is useful to separate the direction of the guidance gradient from its magnitude. Defining

$$\bar{g}_{n,t} := \frac{g_{n,t}}{\|g_{n,t}\|_2}, \tag{4.9}$$

the same velocity modification can equivalently be expressed as

$$\tilde{u}_{n,t} = u_{n,t} - q_{n,t}\bar{g}_{n,t}, \qquad q_{n,t} := \lambda_{n,t}\|g_{n,t}\|_2. \tag{4.10}$$

This representation will be convenient when comparing the methods developed below. Some methods directly control the multiplier of the raw gradient, whereas others naturally control the magnitude of the resulting intervention. Equations (4.7) and (4.10) are equivalent representations of the same update.

4.1.3 Guidance-strength parameterization

We decompose the raw-gradient multiplier introduced in Equation (4.7) as

$$\lambda_{n,t} = w_n \, \gamma_t \, \nu_{n,t}. \tag{4.11}$$

The three terms play distinct roles. The scalar w_n controls the overall guidance magnitude at weather step n. The profile γ_t controls how this magnitude is distributed over the internal flow. Finally, $\nu_{n,t}$ represents an optional method-specific normalization or scaling factor.

We take γ_t to be a non-negative normalized profile with maximum value one. Only its relative variation across flow time is relevant: any positive multiplicative rescaling of γ_t can be absorbed into w_n. The concrete profile families considered in this thesis are therefore an experimental design choice and are introduced later together with the experimental setup.

The decomposition in Equation (4.11) also clarifies the distinction between the methods developed below. In Optimize-Gain, the flow-time profile γ_t is fixed and only the scalar w_n is optimized. In Optimize-Schedule, the interventions at individual flow steps are instead optimized jointly. In Close-Gap, the profile has a different interpretation altogether: rather than prescribing the relative strength of guidance, it prescribes the desired evolution of the remaining target gap. We make this distinction explicit when deriving each method.

Finally, we do not apply guidance at the last Euler step of the residual flow. For ArchesWeather, the final step size satisfies $h_t = s_t$. Since

$$u_{n,t} = \frac{\hat{r}_{n,t} - z_{n,t}}{s_t}, \tag{4.12}$$

the unguided final Euler update gives

$$z_{n,t+1} = z_{n,t} + s_t \frac{\hat{r}_{n,t} - z_{n,t}}{s_t} = \hat{r}_{n,t}. \tag{4.13}$$

A guidance correction applied at this point would therefore directly alter the final residual without any subsequent model evaluation in which the perturbed state could be processed. We consequently set the guidance magnitude to zero at the final flow step for all methods considered in this chapter.

4.2 Target realization algorithm

The formulation above leaves open how the guidance magnitude should be chosen in order to realize the prescribed target. Since the target $y_n^\star$ is known before generation, we adapt the guidance strength to the target rather than selecting it independently of the desired outcome. Preliminary approaches that motivated the formulation developed here are discussed in Appendix A.

We prescribe the relative distribution of guidance over flow time through a normalized profile γ_t and optimize only its overall magnitude. At weather step n, we therefore set

$$\lambda_{n,t} = w_n \gamma_t, \tag{4.14}$$

such that all flow-time guidance strengths are determined by the single scalar $w_n \geq 0$. Since the target is specified separately at every weather step, w_n is optimized independently for each weather step.

For a fixed initial noise realization, running the complete guided flow defines the final target loss as a scalar function

$$\mathcal{L}_n(w_n) := \xi_n(w_n)^2. \tag{4.15}$$

We then seek a stationary point

$$\frac{\mathrm{d}\mathcal{L}_n}{\mathrm{d}w_n} = 0. \tag{4.16}$$

If the prescribed target can be reached exactly, any $w_n^\star$ satisfying $\xi_n(w_n^\star) = 0$ also satisfies Equation (4.16). If exact closure is not reached, we retain the evaluated value of w_n that produces the smallest remaining squared gap.

The gain w_n influences the terminal loss through the complete sequence of guided Euler steps,

$$w_n \longrightarrow z_{n,1} \longrightarrow z_{n,2} \longrightarrow \cdots \longrightarrow z_{n,T} \longrightarrow \mathcal{L}_n. \tag{4.17}$$

Computing $\mathrm{d}\mathcal{L}_n/\mathrm{d}w_n$ therefore requires propagating the sensitivity of the terminal loss backwards through the flow. Following the decomposed backpropagation strategy of Liu et al. [17], we perform this backward propagation using vector–Jacobian products, without retaining the complete unrolled computation graph.

We express the resulting recursion using the adjoint

$$p_{n,t} := \nabla_{z_{n,t}} \mathcal{L}_n, \tag{4.18}$$

which measures the sensitivity of the final target loss to the noisy state at flow step t. At the endpoint,

$$p_{n,T} = \nabla_{z_{n,T}} \mathcal{L}_n = \nabla_{z_{n,T}} \xi_n^2. \tag{4.19}$$

During each forward evaluation of w_n, the guidance gradients $g_{n,t}$ are recomputed from the corresponding trajectory. During the backward pass, however, we treat these gradients as fixed. Under this frozen-guidance-gradient approximation, the guided Euler update

$$z_{n,t+1} = z_{n,t} + h_t \left(u_{n,t} - \lambda_{n,t} g_{n,t} \right) \tag{4.20}$$

has the approximate state Jacobian

$$\frac{\partial z_{n,t+1}}{\partial z_{n,t}} \approx I + h_t \frac{\partial u_{n,t}}{\partial z_{n,t}}. \tag{4.21}$$

The adjoint can therefore be propagated backwards according to

$$p_{n,t} \approx \left(\frac{\partial z_{n,t+1}}{\partial z_{n,t}} \right)^\top p_{n,t+1} \tag{4.22}$$

$$= p_{n,t+1} + h_t \left(\frac{\partial u_{n,t}}{\partial z_{n,t}} \right)^\top p_{n,t+1}. \tag{4.23}$$

Although the optimization variable is w_n, rather than the states $z_{n,t}$, these adjoints are required because a change in w_n modifies the trajectory at every guided flow step. Using $\lambda_{n,t} = w_n \gamma_t$, the chain rule gives

$$\frac{\mathrm{d}\mathcal{L}_n}{\mathrm{d}w_n} = \sum_{t=0}^{T-2} \frac{\partial \mathcal{L}_n}{\partial \lambda_{n,t}} \frac{\partial \lambda_{n,t}}{\partial w_n} = \sum_{t=0}^{T-2} \gamma_t \frac{\partial \mathcal{L}_n}{\partial \lambda_{n,t}}. \tag{4.24}$$

At flow step t, the direct change of the next noisy state with respect to $\lambda_{n,t}$ is

$$\frac{\partial z_{n,t+1}}{\partial \lambda_{n,t}} = -h_t g_{n,t}. \tag{4.25}$$

Since $p_{n,t+1}$ describes the sensitivity of the final loss to $z_{n,t+1}$, it follows that

$$\frac{\partial \mathcal{L}_n}{\partial \lambda_{n,t}} \approx p_{n,t+1}^\top \frac{\partial z_{n,t+1}}{\partial \lambda_{n,t}} \tag{4.26}$$

$$= -h_t p_{n,t+1}^\top g_{n,t}. \tag{4.27}$$

Substituting this result into Equation (4.24) gives the derivative with respect to the overall guidance gain,

$$\frac{\mathrm{d}\mathcal{L}_n}{\mathrm{d}w_n} \approx -\sum_{t=0}^{T-2} h_t \gamma_t p_{n,t+1}^\top g_{n,t}. \tag{4.28}$$

The derivative therefore combines two quantities at each flow step: $g_{n,t}$ describes how guidance locally changes the noisy state, while $p_{n,t+1}$ describes how a change to that state propagates to the final target loss.

Because the resulting optimization problem is one-dimensional, we solve Equation (4.16) using the secant method. Given two previous evaluations $w_n^{(j-1)}$ and $w_n^{(j)}$, the next estimate is

$$w_n^{(j+1)} = w_n^{(j)} - \frac{\mathcal{L}_n'(w_n^{(j)})(w_n^{(j)} - w_n^{(j-1)})}{\mathcal{L}_n'(w_n^{(j)}) - \mathcal{L}_n'(w_n^{(j-1)})}. \tag{4.29}$$

All evaluations use the same initial noise realization. Consequently, $\mathcal{L}_n(w_n)$ changes only as a result of the guidance gain rather than because of stochastic differences between repeated samples.

The resulting method restricts the guidance schedule to the one-dimensional family

$$\boldsymbol{\lambda}_n = w_n \boldsymbol{\gamma}. \tag{4.30}$$

This retains control over the relative placement of guidance along the flow while reducing target realization to the optimization of a single scalar at each weather step. The guidance magnitude is therefore adapted to the prescribed target rather than chosen beforehand. The initialization of the secant search, stopping criteria, numerical safeguards, and maximum

number of evaluations are specified together with the experimental setup.

Chapter 5

Evaluation

We evaluate our guidance method against the two requirements we set for it: *achievement*, the ability to reach the specified target, and *realism*, the physical plausibility of the states it generates.

The first requirement concerns whether the algorithm reaches the target, and how well it does so across settings. We test this through convergence, stability, and trajectory deviation, which together expose the failure modes of the algorithm and reveal which design choices are more adequate overall.

The second requirement concerns whether the generated states are realistic, in terms of temporal, spatial, vertical, and cross-variable coherence.

The evaluation is organised as a funnel. In *flow-time* we assess the sampling dynamics of the algorithm under different guidance schedules, and select the most appropriate one through the defined tests. In *weather-time* we then characterise the parameters of the experimental setting, select the settings that produce the most desirable results, and finally validate the realism of the generated trajectories. Chapter 6 carries out each stage in turn and reports the concrete settings and results; this chapter defines only *what* we test and *how* we score it.

—this is part of the discussion maybe, if not it has to be reframed in a more specific way Alongside these tests, the discussion examines further aspects that the framework does not measure directly—such as the diversity of the generated trajectories and the informativeness of the gradient—but that bear on the overall value of the method and its applications.

5.1 Flow-time tests

Here we assess the algorithm under different settings, internal hyperparameters, and most importantly under different guidance schedules, with the purpose of selecting the most adequate parameters for the weather-time experiments. Before asking whether the method produces realistic results, we first have to establish that it works: that it reaches the target, does so reliably across settings, and does so in a desirable manner (stability and deviation concerns).

We test the achievement requirement and well–behaveheadness of the trajectory for different guidance algorithms through the following tests:

T1 – Reliability
The gap to the target is driven to zero. We report the mean and standard deviation of the final gap.

T2 – Stability
The trajectory is free of oscillatory behaviour. We measure this simply with respect to the pushback measured in the average trajectory (gui vs. ung steps) and whether the algorithm overshoots.

T3 – Closeness
We check that the guided trajectory does not deviate excessively from the unguided one. We report the total amount of guidance needed, and the pathlength in PCA latent space, and the cosine similarity between the steps, compared to the gui-ung run.

TODO: set a bad initialization on purpose

5.2 Weather-time assessment

In weather-time we first characterise the parameters of the experimental setting—to describe their properties and to select settings for the larger experiments—and then validate the realism of the resulting trajectories. Throughout, we pay special attention to the diversity of the produced results as a secondary property of interest, alongside realism.

5.2.1 Parameter characterisation tests

In the following we provide a set of qualitative tests to better understand the role of the main parameters of the guidance at weather time. We test parameters in isolation, using the

remaining parameters as controls. We proceed in three stages, examining in turn the region of interest, the variable of interest, and the intervention profile. For each we show different charts and metrics to expose what happens across these settings.

— maybe here is where the baseline would shine.

The reason for these tests is mostly to enhance our intuition about the role of each parameter, and provide a guideline on to select parameters for the large use cases we aim to produce with this work.

Region of interest

Here we assess variability of results by changing size, shift (from center), and coast-land covering of region of the interest.

Variable of interest

For simplicity we focus on heatwaves, and therefore on surface temperature. To probe the informativeness of the gradient, in this test we push the most correlated variables in an attempt to reproduce the pattern obtained by pushing temperature directly.

Intervention profile

Here we assess the degradation of the target patterns across intensity profiles. The intuition is that at some point things should brake down in some trivial way.

5.2.2 Realism tests

Once the guidance method and its parameters are selected, we assess the realism of the generated trajectories. Realism is central to this work, because we deliberately guide the generative model, possibly outside its training distribution. The tests themselves, however, are not specific to this work: they concern the general problem of evaluating the coherence of data-driven models, namely that the generated states should be consistent with the ground-truth dataset in both the guided and unguided settings.

We assess coherence to the ground-truth dataset along four dimensions:

T4 (Temporal).
Transitions of a variable across weather steps should be smooth.

T5 (Spatial).
Patterns over the region of interest should be spatially coherent.

T6 (Vertical).
Patterns should be coherent across pressure levels. We test this using a level rank coherence metric.

T7 (Cross-variable).
Patterns should be coherent with the learned weather transition function across variables. However, this is difficult to test quantitatively, we thus only make a qualitative assessment. (TODO: use the expert knowledge here)

We benchmark these properties against the ground-truth data as anchor (reference set), and against the unguided forecast as baseline, comparing unguided, guided-minus-unguided, and guided states. For T4–T6 we compute difference maps along the axis of interest and report the min, mean, max, and standard deviation across maps. For T7 we compute the cross-variable distributions over the weather steps of the ground-truth data, and report the deviation of the unguided, guided-minus-unguided, and guided states from their average.

For all four tests we average over the experiments and report the mean and standard deviation across runs. This indicates whether the generated extremes are faithful to the data distribution. A subtle but important point: for this to be meaningful, the base forecast must have missed the potential heat spots. The extreme is not only about the average—which the forecast most likely covers faithfully—but about the extreme spots, so both the mean and the max must be tested.

Chapter 6 sets out the concrete settings for each stage of this framework and reports the results, which Chapter 7 then discusses.

5.3 Method Validation

This approach can be used to guide any state towards an extreme over the specified mask.

To validate that the method is doing something sensible we can retrieve real heatwaves from the test dataset (everything that is not included in the train set), and observe what the model is doing.

Another way would be to guide it towards the ground truth and observe the RMSE of other variables compared to the ung.

Validate method against: gt, opposite direction, same direction. Make sure you apply same amount of guidance needed when definining y star.

Chapter 6

Experiments

This chapter carries out the evaluation framework of Chapter 5. Each section follows the same pattern: we first state the concrete settings, then report the results and the decision they lead to. Tests are referenced by the identifiers introduced in Chapter 5 (T1–T7). —actually reference the latex identifiers like for equations

6.1 Flow-time evaluation

6.1.1 Setup

We first investigate how the placement of guidance within the generative flow affects target realization. To isolate the effect of flow time, we keep the target region and intensity fixed while varying the atmospheric starting state and stochastic realization. We use the following setup:

- one fixed target mask over Europe;
- three different atmospheric starting states;
- $N = 1$ weather step and $M = 3$ ensemble members;
- one target intensity of $+0.25\%$;
- three single-step guidance profiles: `spike@1`, `spike@3`, `spike@6`, `spike@12`, `spike@24`.

Each spike profile applies guidance at only one flow step, allowing early, intermediate, and late interventions to be compared directly. The guidance gain w_n is optimized independently for each experiment using the target-realization algorithm described in Section 4.2. The resulting design comprises $3 \times 3 \times 5 = 45$ guided experiments.

Given the results of this first test, we now test something related. The question is then not only during which phase of the guidance it's most appropriate to guide, but whether to spread the guidance over multiple steps would be benefitial. We test this by defining a_t to be interpolated between spike 1-3, 3-6, ...

Of course optimizing this would be most appropriate. What we are trying to do here is to observe empirically whether some properties emerge before defering future research to the development of an appropriate algorithm, which could use this information for initialization.

6.1.2 Achievement tests

The difficulty of tuning the first and second algorithm make them less suitable than the third one. Regarding algorithm 1, having a lower computational complexity is useless in practice if we then have to perform multiple costly runs. Regarding algorithm 2, having a desirability requirement encoded in the loss itself only complicate things. Tuning the optimization of this algorithms outweights the benefits, and induces a much higher cost in practical terms to the algorithm.

At later stages, the gradient becomes more similar to the mask (is more diluted, less sparse), which makes the guided pattern at the end of the flow be more spread.

Using a spike@1 approach is essentially equivalent to shifting the starting point of the flow trajectory (especially for the region over the mask) and let particle flow across the learned ODE.

What we observe is that the model doesn't necessarily pushes back against us. In contrary, when the method is tuned well, the model will do most of the pushing itself (look at this in percentage).

6.2 Weather-time evaluation

Throughout the weather-time experiments we vary the start state while fixing $N = 2$ and $M = 3$: this controls for the effects of $N = 1$ and for seed-specific effects, while keeping computational and memory requirements as low as possible.

Note that we will use the same experiments for the parameter characterization tests and for the realism tests.

— the realism tests could simply be used in each parameter subsection

6.2.1 Parameter characterisation tests

Region of interest

Variable of interest

Intervention profile

6.2.2 Realism tests

6.3 Use cases

For the final use cases we fix the parameters selected above and increase N and M, producing the experiments we had in mind when setting the goals of the project.

6.3.1 Europe 2020-06-01

6.3.2 Australia 2020-01-01

6.3.3 North America 2020-07-01

Chapter 7

Discussion

7.1 Mask

We experimented with a gaussian mask, but realized in our experiments that the best thing to do is equal weighting inside the region. The correct way to define the region of interest is to choose a greater mask size instead of using some weighting scheme fading out of an epicenter. The mask size should be chosen to cover the entire area of interest, ensuring that all relevant data is included in the analysis. A larger mask size can help to reduce noise and improve the accuracy of the results, while a smaller mask size may exclude important information. It is important to carefully consider the appropriate mask size for each specific application to achieve optimal results. Either way we (most probably) loose the global coherence for the guided weather states, so we might as well define directly the region of interest with a mask. Additionally, this enables us to actually make statements about "the region of interest", having equal weights everywhere. Introducing different weigths might bias the generation in a way that is not desired.

We will see that including or not a specific area will change the guided patterns substantially. This however is consistent with the notion of "region of interest". If we consider a greater area for the changes, the changes will be applied respectively.

— note here: it may also be that the pushed area is coherent with the rest, and that the next deterministic prediction influences other regions too It is really important to understand how to evaluate what we are doing here. Since we are aiming at a specific region of interest, and we are guiding mainly locally over the region of interest, loosing the global coherence of the weather states. We shall thus include this in our evaluation by comparing inside region, outside region, and full spatial domain.

7.2 ...

The algorithm doesn't produce diverse results, because the gradients are really stable. This has to do with the fact that we have learned one and only one flow model (transition function). If we had multiple residual diffusion model, we would expect the possible effects to be more diverse. However, the second interpretation might be that the gradient provides with the information about the most likely direction of change (which is true under the learned data distribution).

This happens because the guidance vector is a scaled version of the flow model's gradient wrt to the noisy state. The gradient is stable across the full trajectory, because the only variable inputs are the noise itself and the timestep of the flow. The stochasticity of the noise is not enough to produce diverse results. The direction of the gradient is always the same. If we say "let the masked average go up by phi percent", the only thing that changes is the magnitude of the gradient, but not its direction, which is specified by the Jacobian.

This is in contrast to guidance settings.

Shift between atmosphere and stratosphere.

Put here all observations

Chapter 8

Conclusion

8.1 Summary

8.2 Future Directions

8.3 Conclusive Remarks

Appendices

Appendix A

Development of the target-realization algorithm

Before arriving at the target-realization algorithm presented in Section 4.2, we explored two more flexible strategies for determining the guidance magnitude: Close-Gap and Optimize-Schedule. These preliminary approaches motivated the final choice to prescribe the relative flow-time profile and optimize only a single overall guidance gain.

A.1 Close-Gap

The first approach exploits the fact that the desired target is known before generation by prescribing how the remaining target gap should evolve throughout the flow. Let

$$\bar{\xi}_{n,t} := \gamma_t^{\text{gap}}\, \xi_{n,0}, \tag{A.1}$$

denote the desired remaining gap at flow step t, where γ_t^{gap} specifies the fraction of the initial gap that should remain.

At each flow step, the method computes the local displacement of the noisy state required to move the current gap $\xi_{n,t}$ towards $\bar{\xi}_{n,t}$. Using a first-order Taylor approximation,

$$\xi_{n,t}(z_{n,t} + \Delta z_{n,t}) \approx \xi_{n,t}(z_{n,t}) + \left(\nabla_{z_{n,t}} \xi_{n,t}\right)^\top \Delta z_{n,t}. \tag{A.2}$$

Requiring the corrected gap to equal the prescribed value gives

$$\left(\nabla_{z_{n,t}}\xi_{n,t}\right)^{\top}\Delta z_{n,t} = \bar{\xi}_{n,t} - \xi_{n,t}. \tag{A.3}$$

Among the displacements satisfying this local constraint, the smallest in Euclidean norm is obtained by moving along the gradient of the gap,

$$\Delta z^{\star}_{n,t} = \frac{\bar{\xi}_{n,t} - \xi_{n,t}}{\left\|\nabla_{z_{n,t}}\xi_{n,t}\right\|_2^2}\nabla_{z_{n,t}}\xi_{n,t}. \tag{A.4}$$

Using

$$g_{n,t} = \nabla_{z_{n,t}}\xi^2_{n,t} = 2\xi_{n,t}\nabla_{z_{n,t}}\xi_{n,t}, \tag{A.5}$$

the desired displacement can equivalently be written as

$$\Delta z^{\star}_{n,t} = -\frac{2\xi_{n,t}\left(\xi_{n,t} - \bar{\xi}_{n,t}\right)}{\|g_{n,t}\|_2^2}g_{n,t}. \tag{A.6}$$

The guidance contribution to the Euler update is

$$\Delta z^{\mathrm{gui}}_{n,t} = -h_t\lambda_{n,t}g_{n,t}. \tag{A.7}$$

Requiring this contribution to realize the desired local displacement yields

$$\lambda^{\mathrm{CG}}_{n,t} = \frac{2\xi_{n,t}\left(\xi_{n,t} - \bar{\xi}_{n,t}\right)}{h_t\|g_{n,t}\|_2^2}. \tag{A.8}$$

The attraction of this formulation is its efficiency: the guidance magnitude is determined online and only a single flow is required. However, the update is greedy. It computes the intervention required to satisfy the prescribed gap under a local approximation, without accounting for the subsequent displacement induced by the learned velocity field. The resulting trajectory can therefore overshoot the prescribed gap and require compensating corrections at later flow steps.

A slower gap-closing profile can reduce this behaviour, but then introduces a schedule parameter controlling how aggressively the gap is closed. Choosing this parameter requires repeated runs, reducing the practical advantage of the otherwise single-pass method. For this reason, we next considered optimizing the flow-time intervention schedule directly.

A.2 Optimize-Schedule

Rather than prescribing the evolution of the target gap, Optimize-Schedule treats the intervention magnitudes across the complete flow as optimization variables.

We parameterize the intervention through its contribution to the Euler update,

$$k_{n,t} := h_t \lambda_{n,t}, \tag{A.9}$$

such that

$$z_{n,t+1} = z_{n,t} + h_t u_{n,t} - k_{n,t} g_{n,t}. \tag{A.10}$$

The quantity $k_{n,t} g_{n,t}$ is the state displacement induced directly by guidance at flow step t. We optimize the complete intervention trajectory

$$\mathbf{k}_n = (k_{n,0}, \ldots, k_{n,T-2}) \tag{A.11}$$

using the objective

$$J_n(\mathbf{k}_n) = \xi_n^2 + \kappa \sum_{t=0}^{T-2} \|k_{n,t} g_{n,t}\|_2^2 . \tag{A.12}$$

The first term measures target realization, while the second penalizes the total squared magnitude of the guidance-induced state displacements. This allows the optimization to prefer schedules that approach the prescribed target while limiting the amount of intervention applied along the flow.

In principle, this formulation removes the need to prescribe a gap-closing trajectory and allows the temporal distribution of guidance to adapt freely. In practice, however, the optimization is strongly coupled. Changing $k_{n,t}$ modifies the noisy trajectory, which changes the clean-state estimates and therefore also the guidance gradients encountered at subsequent flow steps. Consequently, the optimal intervention at one flow step depends on the interventions applied throughout the trajectory.

In preliminary experiments, this high-dimensional optimization was strongly sensitive to its initialization and could converge rapidly to different local solutions. The additional flexibility therefore came at the cost of a substantially more difficult optimization problem.

These observations motivated the lower-dimensional formulation adopted in the main method.

Instead of determining an independent intervention magnitude at every flow step, we prescribe a normalized profile γ_t and restrict the guidance schedule to

$$\lambda_{n,t} = w_n \gamma_t. \tag{A.13}$$

Target realization then reduces to optimizing the single scalar w_n at each weather step, as described in Section 4.2.

Bibliography

[1] Arpit Bansal et al. "Universal Guidance for Diffusion Models". In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops*. 2023, pp. 843–852.

[2] Heli Ben-Hamu et al. "D-Flow: Differentiating through Flows for Controlled Generation". In: *arXiv preprint arXiv:2402.14017* (2024). DOI: 10.48550/arXiv.2402.14017. URL: https://arxiv.org/abs/2402.14017.

[3] Kaifeng Bi et al. "Accurate Medium-Range Global Weather Forecasting with 3D Neural Networks". In: *Nature* 619 (2023), pp. 533–538. DOI: 10.1038/s41586-023-06185-3.

[4] Noah D. Brenowitz et al. "Climate in a Bottle: Towards a Generative Foundation Model for the Kilometer-Scale Global Atmosphere". In: *arXiv preprint arXiv:2505.06474* (2025). DOI: 10.48550/arXiv.2505.06474. URL: https://arxiv.org/abs/2505.06474.

[5] Hyungjin Chung et al. "Diffusion Posterior Sampling for General Noisy Inverse Problems". In: *arXiv preprint arXiv:2209.14687* (2023). DOI: 10.48550/arXiv.2209.14687. URL: https://arxiv.org/abs/2209.14687.

[6] Guillaume Couairon et al. "ArchesWeather & ArchesWeatherGen: A Deterministic and Generative Model for Efficient ML Weather Forecasting". In: *arXiv preprint arXiv:2412.12971* (2024). DOI: 10.48550/arXiv.2412.12971.

[7] Prafulla Dhariwal and Alexander Nichol. "Diffusion Models Beat GANs on Image Synthesis". In: *Advances in Neural Information Processing Systems*. Vol. 34. 2021, pp. 8780–8794.

[8] Patrick Esser et al. "Scaling Rectified Flow Transformers for High-Resolution Image Synthesis". In: *Proceedings of the 41st International Conference on Machine Learning*. Vol. 235. Proceedings of Machine Learning Research. PMLR, 2024, pp. 12606–12633.

[9] Kanxuan He and Hongshan Guo. "Probabilistic Sequence Generation Guided by Intensity–Duration Extreme Profiles". In: *SPIGM Workshop at ICML*. 2026. URL: https://openreview.net/forum?id=n8bQJs1dip.

[10] Hans Hersbach. "Decomposition of the Continuous Ranked Probability Score for Ensemble Prediction Systems". In: *Weather and Forecasting* 15.5 (2000), pp. 559–570. DOI: 10.1175/1520-0434(2000)015<0559:DOTCRP>2.0.CO;2.

[11] Hans Hersbach et al. "The ERA5 global reanalysis". In: *Quarterly Journal of the Royal Meteorological Society* 146.730 (2020), pp. 1999–2049. DOI: 10.1002/qj.3803.

[12] Jonathan Ho and Tim Salimans. "Classifier-Free Diffusion Guidance". In: *arXiv preprint arXiv:2207.12598* (2022). DOI: 10.48550/arXiv.2207.12598.

[13] Zhanxiang Hua et al. "Weather Prediction with Diffusion Guided by Realistic Forecast Processes". In: *arXiv preprint arXiv:2402.06666* (2024). DOI: 10.48550/arXiv.2402.06666. URL: https://arxiv.org/abs/2402.06666.

[14] Remi Lam et al. "Learning Skillful Medium-Range Global Weather Forecasting". In: *Science* 382.6677 (2023), pp. 1416–1421. DOI: 10.1126/science.adi2336.

[15] Amaury Lancelin et al. "AI-Boosted Rare Event Sampling to Characterize Extreme Weather". In: *arXiv preprint arXiv:2510.27066* (2025). DOI: 10.48550/arXiv.2510.27066. URL: https://arxiv.org/abs/2510.27066.

[16] Yaron Lipman et al. "Flow Matching for Generative Modeling". In: *International Conference on Learning Representations (ICLR)*. 2023.

[17] Xingchao Liu et al. "FlowGrad: Controlling the Output of Generative ODEs With Gradients". In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 2023, pp. 24335–24344.

[18] Frieder Loer, Maybritt Schillinger, and Sebastian Sippel. "Probabilistic Storyline Attribution Using Machine Learning". In: *arXiv preprint arXiv:2606.02550* (2026). DOI: 10.48550/arXiv.2606.02550. URL: https://arxiv.org/abs/2606.02550.

[19] Alexander Quinn Nichol et al. "GLIDE: Towards Photorealistic Image Generation and Editing with Text-Guided Diffusion Models". In: *Proceedings of the 39th International Conference on Machine Learning*. Vol. 162. Proceedings of Machine Learning Research. PMLR, 2022, pp. 16784–16804.

[20] Ilan Price et al. "Probabilistic Weather Forecasting with Machine Learning". In: *Nature* 637.8044 (2025), pp. 84–90. DOI: 10.1038/s41586-024-08252-9.

[21] Stephan Rasp et al. "WeatherBench 2: A Benchmark for the Next Generation of Data-Driven Global Weather Models". In: *Journal of Advances in Modeling Earth Systems* 16.6 (2024), e2023MS004019. DOI: 10.1029/2023MS004019.

[22] Yifei Shen et al. "Understanding and Improving Training-Free Loss-Based Diffusion Guidance". In: *Advances in Neural Information Processing Systems*. Vol. 37. 2024. DOI: 10.52202/079017-3460.

[23] Theodore G. Shepherd et al. "Storylines: An Alternative Approach to Representing Uncertainty in Physical Aspects of Climate Change". In: *Climatic Change* 151.3–4 (2018), pp. 555–571. DOI: 10.1007/s10584-018-2317-9.

[24] Jiaming Song et al. "Loss-Guided Diffusion Models for Plug-and-Play Controllable Generation". In: *Proceedings of the 40th International Conference on Machine Learning*. Vol. 202. Proceedings of Machine Learning Research. PMLR, 2023, pp. 32483–32498.

[25] Vikki Thompson et al. "High Risk of Unprecedented UK Rainfall in the Current Climate". In: *Nature Communications* 8 (2017), p. 107. DOI: 10.1038/s41467-017-00275-3.

[26] Ayumu Ueyama, Kazuhiko Kawamoto, and Hiroshi Kera. "Guided Diffusion Sampling for Precipitation Forecast Interventions". In: *arXiv preprint arXiv:2605.14317* (2026). DOI: 10.48550/arXiv.2605.14317. URL: https://arxiv.org/abs/2605.14317.

[27] Luran Wang et al. "Training Free Guided Flow-Matching with Optimal Control". In: *The Thirteenth International Conference on Learning Representations*. 2025. URL: https://openreview.net/forum?id=61ss5RA1MM.

[28] Tim Whittaker and Alejandro Di Luca. "Pushing the Limits of Extreme Weather: Constructing Extreme Heatwave Storylines with Differentiable Climate Models". In: *arXiv preprint arXiv:2506.10660* (2025). DOI: 10.48550/arXiv.2506.10660.

[29] World Meteorological Organization. *Atlas of Mortality and Economic Losses from Weather, Climate and Water-Related Hazards (1970–2021)*. World Meteorological Organization, 2023.

Eidesstattliche Erklärung

Der/Die Verfasser/in erklärt an Eides statt, dass er/sie die vorliegende Arbeit selbständig, ohne fremde Hilfe und ohne Benutzung anderer als die angegebenen Hilfsmittel angefertigt hat. Die aus fremden Quellen (einschliesslich elektronischer Quellen) direkt oder indirekt übernommenen Gedanken sind ausnahmslos als solche kenntlich gemacht. Die Arbeit ist in gleicher oder ähnlicher Form oder auszugsweise im Rahmen einer anderen Prüfung noch nicht vorgelegt worden.

.....	.....
Ort, Datum	Unterschrift des/der Verfassers/in